Unsupervised Learning

Finding Structure Without Labels

Mehmet Mercangöz

AI and AIS Summer School July/Aug 2026

Lecture Map

1. **Foundations**
   - unlabelled data, structure, similarity, representation
2. **Clustering methods**
   - k-means, hierarchical clustering, DBSCAN, Gaussian mixtures
3. **Dimensionality reduction**
   - PCA, projections, Q and T^2 process monitoring metrics
4. **Representation learning**
   - autoencoders and variational autoencoders
5. **Generative modelling**
   - GANs, generator-discriminator training, limitations

What Is Unsupervised Learning?

Unsupervised learning uses data where the desired answer is **not** provided.

Each observation contains inputs only:

$$\boldsymbol{x}_i \qquad \text{but no target } y_i.$$

The goal is to discover useful structure in the data:

unlabelled data $\boldsymbol{x}_1, \dots, \boldsymbol{x}_n$ → unsupervised learning algorithm → groups, patterns, latent variables, compressed features

Core idea

The learner is not told the correct answer. It must infer structure from the geometry and statistics of the data.

Supervised and Unsupervised Learning

Setting	**Data**	**Typical question**
Supervised learning	Inputs with known targets	Can we predict the target for a new input?
Unsupervised learning	Inputs without targets	Are there patterns, groups, or simpler coordinates?
Reinforcement learning	States, actions, rewards	Which actions should be taken over time?

This lecture focuses on **unsupervised learning**.

The Unsupervised Learning Dataset

Suppose we have n observations and p features.

$$X = \begin{bmatrix} - - - \boldsymbol{x}_1^\top - - - \\ - - - \boldsymbol{x}_2^\top - - - \\ \vdots \\ - - - \boldsymbol{x}_n^\top - - - \end{bmatrix} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}.$$

- Each **row** is one observation.
- Each **column** is one feature.
- There is no separate target vector $\boldsymbol{y}$.

What Can We Learn Without Labels?

Find groups

- customer segments;
- operating regimes;
- fault modes;
- document topics.

Learn representations

- embeddings;
- latent variables;
- denoised signals;
- anomaly scores.

Reduce dimension

- visualise high-dimensional data;
- remove redundancy;
- compress features.

Generate new samples

- synthetic images;
- scenario generation;
- data augmentation.

A Running Example: Operating Data

Imagine historical data from a process unit, but no labels.

Observation	**Flow**	**Temp.**	**Pressure**	**Energy**
1	102	81.0	2.4	52.1
2	98	80.4	2.3	50.7
3	141	93.2	3.1	71.8
4	138	92.6	3.0	69.9
5	116	85.1	2.6	57.4

Possible unsupervised questions:

- Are there distinct operating regimes?
- Can we visualise the data in two coordinates?
- Can we learn a compressed representation of the process state?
- Can we detect observations that do not look normal?

Preprocessing Matters

Many unsupervised methods depend strongly on distances and variances.

A common standardisation is

$$x_{ij}^{\text{scaled}} = \frac{x_{ij} - \mu_j}{s_j},$$

where μ_j and s_j are computed from the training data.

Why this matters

If temperature is measured in degrees, pressure in bar, and flow in tonnes/hour, the largest numerical scale may dominate the result.

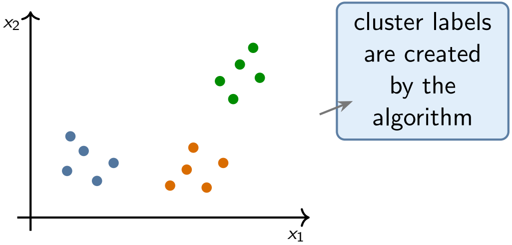
For clustering and PCA, the choice of preprocessing is part of the model.

What Is Clustering?

Clustering assigns observations to groups using only the input features.

$$\boldsymbol{x}_i \longmapsto \text{cluster label } c_i.$$

But these cluster labels are not given by a teacher. They are constructed by the algorithm.



Important

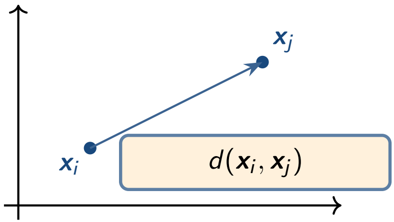
A cluster is not automatically a meaningful class. It depends on the chosen similarity measure.

Similarity and Distance

Most clustering methods need a way to compare observations.

For numerical features, a common choice is Euclidean distance:

$$d(\boldsymbol{x}_i, \boldsymbol{x}_j) = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2.$$



- Distance-based clustering is sensitive to feature scaling.
- Different distances can lead to different clusters.

k-Means Clustering: Objective

k-means represents each cluster by a centroid $\boldsymbol{\mu}_k$.

The objective is to minimise within-cluster squared distances:

$$J = \sum_{i=1}^{n} \|\boldsymbol{x}_i - \boldsymbol{\mu}_{c_i}\|_2^2 .$$

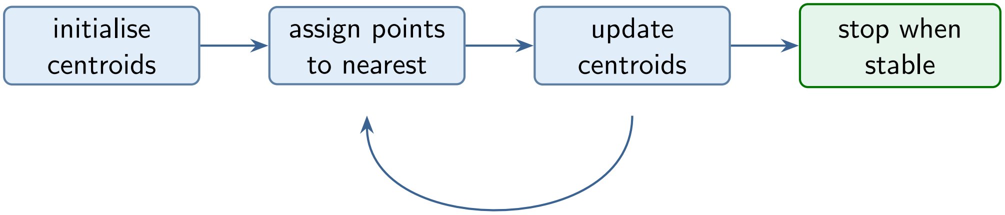
- c_i is the cluster assigned to observation i.
- $\boldsymbol{\mu}_{c_i}$ is the centroid of that cluster.
- The number of clusters k is chosen before running the algorithm.

Interpretation

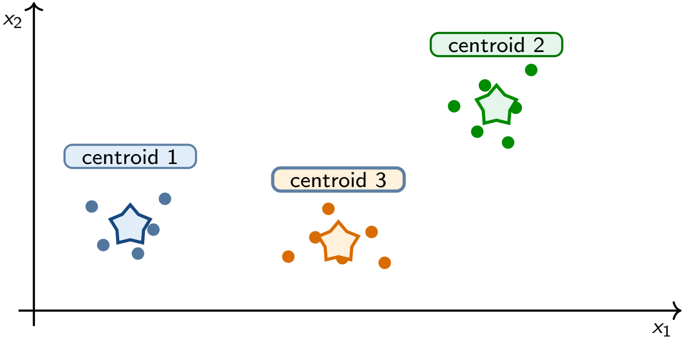
k-means tries to place k representative points so that each data point is close to its representative.

k-Means Algorithm

1. Choose the number of clusters k.
2. Initialise k centroids.
3. Assign each point to its nearest centroid.
4. Update each centroid as the mean of the assigned points.
5. Repeat assignment and update until the assignments stop changing.



k-Means: Visual Picture



Each centroid is the average location of the points assigned to that cluster.

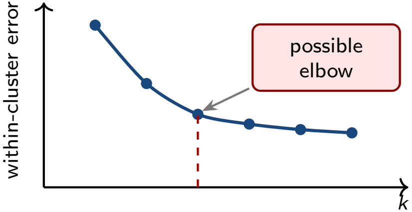
Choosing the Number of Clusters

For k-means, the number of clusters k must be chosen.

Elbow idea

Run k-means for several values of k and plot the within-cluster sum of squares.

Choose a value where the improvement begins to flatten.



Caution

There may not be a clear elbow. The "right" number of clusters often depends on the purpose.

Limitations of k-Means

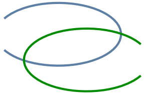
k-means is simple and useful, but it has strong assumptions.

Works best when

- clusters are roughly round;
- clusters have similar size;
- Euclidean distance is meaningful;
- k is known or easy to choose.

Can struggle when

- clusters are curved;
- clusters have different densities;
- outliers are present;
- initial centroids are poor.



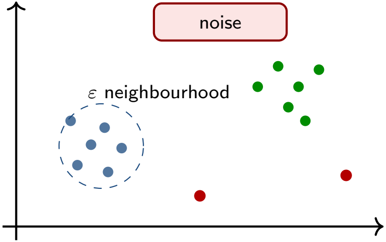
Curved clusters can be difficult for k-means.

DBSCAN: Density-Based Clustering

DBSCAN finds dense regions separated by sparse regions.

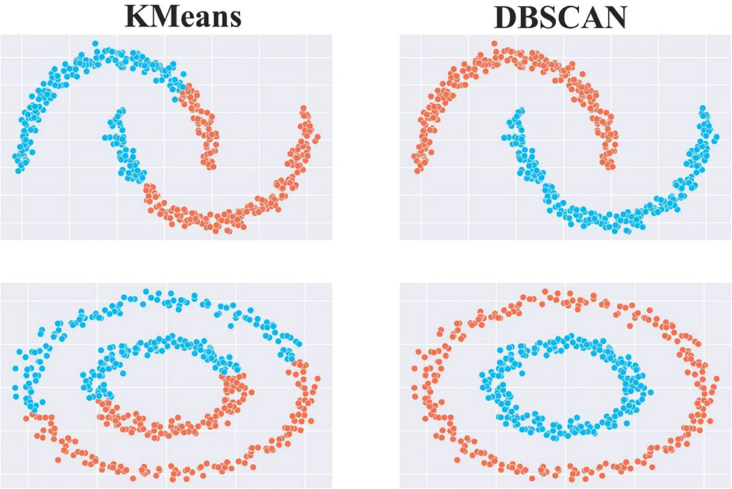
Two key parameters

- ε: neighbourhood radius;
- minPts: minimum number of neighbours for a dense region.



- Can identify outliers as noise.
- Can find non-spherical clusters.
- Does not require choosing k directly.

k-Means vs DBSCAN

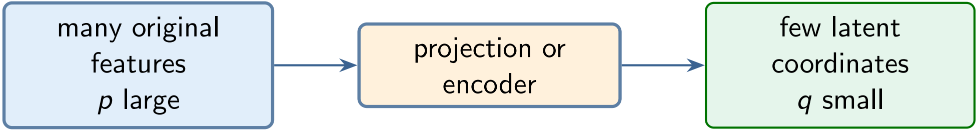


Why Dimensionality Reduction?

High-dimensional data can be difficult to interpret and visualise.

Dimensionality reduction maps

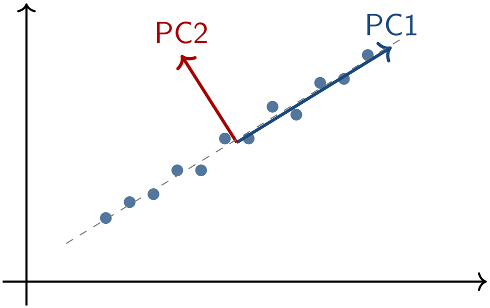
$$\boldsymbol{x}_i \in \mathbb{R}^p \qquad \longmapsto \qquad \boldsymbol{z}_i \in \mathbb{R}^q, \qquad q < p.$$



- Visualisation: reduce to 2 or 3 coordinates.
- Compression: store fewer numbers.
- Feature engineering: remove redundancy before another model.

PCA: The Main Idea

Principal component analysis finds new axes that capture as much variance as possible.



Geometric view

PC1 is the direction along which the projected data have the largest variance. PC2 is the next orthogonal direction.

Step 1: Centre the Data

Suppose X contains n observations and p variables:

$$X = \begin{bmatrix} \boldsymbol{x}_1^\top \\ \boldsymbol{x}_2^\top \\ \vdots \\ \boldsymbol{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times p}.$$

Each row represents one observation:

$$\boldsymbol{x}_i \in \mathbb{R}^p.$$

The sample mean is

$$\bar{\boldsymbol{x}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{x}_i = \frac{1}{n} X^\top \mathbf{1}_n.$$

Centre each observation:

$$\tilde{\boldsymbol{x}}_i = \boldsymbol{x}_i - \bar{\boldsymbol{x}}.$$

In matrix form,

$$\boxed{\tilde{X} = X - \mathbf{1}_n \bar{\boldsymbol{x}}^\top}$$

where $\mathbf{1}_n \in \mathbb{R}^n$ is a vector of ones.

PCA describes variation around the sample mean. After centring, every variable has zero sample mean:

$$\frac{1}{n}\tilde{X}^\top \mathbf{1}_n = \mathbf{0}.$$

Step 2: Covariance Matrix

For centred data, the sample covariance matrix is

$$S = \frac{1}{n-1}\tilde{X}^\top \tilde{X}.$$

- Diagonal entries: variances of individual features.
- Off-diagonal entries: covariances between pairs of features.

For two features:

$$S = \begin{bmatrix} s_{11} & s_{12} \\ s_{12} & s_{22} \end{bmatrix}.$$

PCA uses eigenvectors of the covariance matrix.

Eigenvectors give principal directions; eigenvalues give variances along those directions.

PCA Eigenvalue Problem

PCA solves

$$S\boldsymbol{v}_j = \lambda_j \boldsymbol{v}_j.$$

Quantity	Meaning
$\boldsymbol{v}_j$	principal component direction or loading vector
λ_j	variance captured along direction $\boldsymbol{v}_j$
$\boldsymbol{z}_{ij} = \boldsymbol{v}_j^\top \tilde{\boldsymbol{x}}_i$	score of observation i on principal component j

The eigenvalues are usually sorted as

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p.$$

Projection onto Principal Components

Let V_q contain the first q principal directions:

$$V_q = \begin{bmatrix} \boldsymbol{v}_1 & \boldsymbol{v}_2 & \cdots & \boldsymbol{v}_q \end{bmatrix}.$$

The low-dimensional scores are

$$Z = \tilde{X} V_q.$$

For one point:

$$\boldsymbol{z}_i = V_q^\top \tilde{\boldsymbol{x}}_i.$$

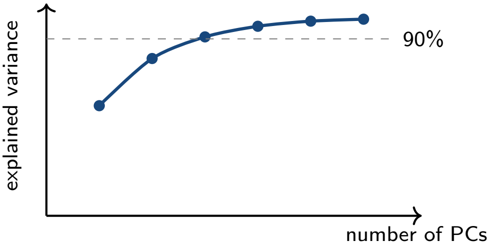
Interpretation

PCA replaces the original features by coordinates along the dominant directions of variation.

Explained Variance

The fraction of variance explained by the first q principal components is

$$\frac{\lambda_1 + \lambda_2 + \cdots + \lambda_q}{\lambda_1 + \lambda_2 + \cdots + \lambda_p}.$$



Reconstruction from PCA

After projecting down to q dimensions, we can approximately reconstruct the data:

$$\hat{X} = ZV_q^\top + \mathbf{1}\bar{\boldsymbol{x}}^\top.$$

For one observation:

$$\hat{\boldsymbol{x}}_i = V_q \boldsymbol{z}_i + \bar{\boldsymbol{x}}.$$

Compression view

If $q \ll p$, PCA stores each observation using fewer numbers, while preserving much of the variance.

From PCA to Process Monitoring

In process monitoring, PCA is usually trained using data from **normal operation**.

The PCA model learns the main correlation structure between process variables:

temperature, pressure, flow, composition, current, vibration, ...

normal operation training data → fit PCA model → new online measurement → monitor T^2, Q

Core idea

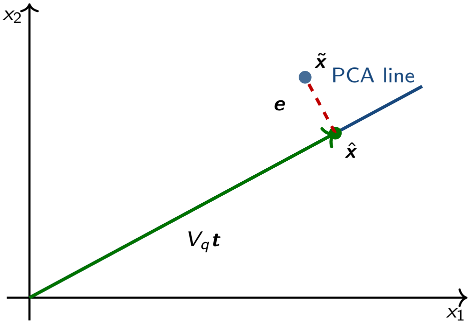
A new sample is normal if it is consistent with the learned PCA subspace and is well reconstructed by it.

PCA Splits a Measurement into Two Parts

For a new centred and scaled process measurement $\tilde{\boldsymbol{x}}$, PCA gives

$$\tilde{\boldsymbol{x}} = \underbrace{V_q \boldsymbol{t}}_{\text{modelled part}} + \underbrace{\boldsymbol{e}}_{\text{residual part}} .$$

$$\boldsymbol{t} = V_q^\top \tilde{\boldsymbol{x}}, \qquad \hat{\boldsymbol{x}} = V_q \boldsymbol{t}, \qquad \boldsymbol{e} = \tilde{\boldsymbol{x}} - \hat{\boldsymbol{x}}.$$



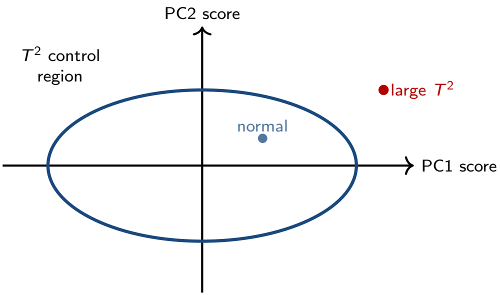
Hotelling's T^2: Distance Inside the PCA Model

The score vector tells us where the sample lies **inside** the PCA subspace:

$$\boldsymbol{t} = \begin{bmatrix} t_1 & t_2 & \cdots & t_q \end{bmatrix}^\top.$$

Hotelling's T^2 scales each score by the variance of its component:

$$\boxed{T^2 = \boldsymbol{t}^\top \Lambda_q^{-1} \boldsymbol{t} = \sum_{j=1}^{q} \frac{t_j^2}{\lambda_j}}$$



Interpretation

Large T^2 means the sample is extreme compared with normal scores, even if it still follows the learned correlation pattern.

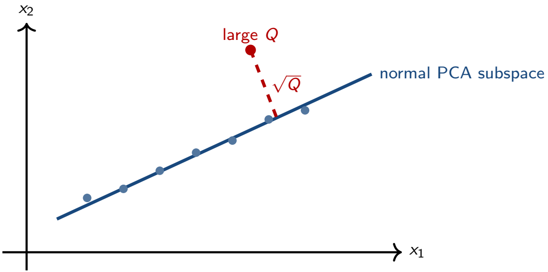
The Q Statistic: Distance Outside the PCA Model

The residual vector measures what PCA could **not** explain:

$$\boldsymbol{e} = \tilde{\boldsymbol{x}} - \hat{\boldsymbol{x}}.$$

The Q statistic, also called squared prediction error (SPE), is

$$\boxed{Q = \|\boldsymbol{e}\|_2^2 = \|\tilde{\boldsymbol{x}} - \hat{\boldsymbol{x}}\|_2^2}$$



Interpretation

Large Q means the sample does not obey the usual correlation structure and is poorly reconstructed by PCA.

Reading T^2 and Q Together

The two statistics monitor different spaces.

Case	Meaning	Possible process interpretation
low T^2, low Q	close to normal model	normal operation
high T^2, low Q	extreme inside PCA subspace	large but correlated operating change
low T^2, high Q	poor reconstruction	new correlation pattern, sensor fault, local abnormality
high T^2, high Q	extreme and poorly reconstructed	strong abnormal event or fault

Important

An alarm indicates "different from normal operation". It does not by itself identify the root cause.

PCA Monitoring Workflow

Offline: build the monitoring model

1. collect normal-operation data;
2. centre and scale variables;
3. fit PCA and choose q;
4. compute training T^2 and Q values;
5. set warning and alarm limits.

Online: monitor new samples

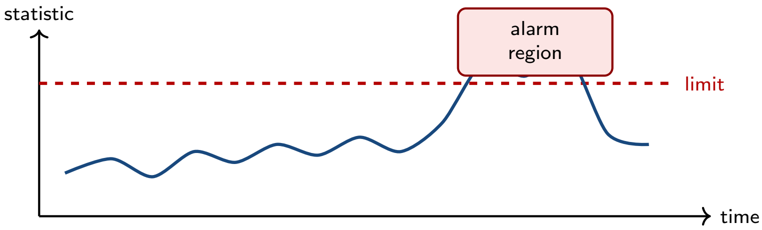
1. centre and scale new measurement;
2. compute score $\boldsymbol{t}$;
3. reconstruct $\hat{\boldsymbol{x}}$;
4. compute T^2 and Q;
5. trigger alarm if limits are exceeded.

new sample $\boldsymbol{x}$ → centre scale → scores $\boldsymbol{t}$ → T^2 Q → alarm?

Control Limits for Monitoring

A PCA monitoring chart compares each statistic with a limit learned from normal operation.

$$T^2(t) \leq T^2_{\text{lim}}, \qquad Q(t) \leq Q_{\text{lim}}.$$



- A simple practical limit is an empirical 95% or 99% percentile from normal training data.
- Parametric limits can also be used when distributional assumptions are reasonable.

Contribution Plots: From Alarm to Diagnosis

After a T^2 or Q alarm, the next question is:

which variables contributed most to the alarm?

Q contribution

The residual contribution of variable j is often inspected using

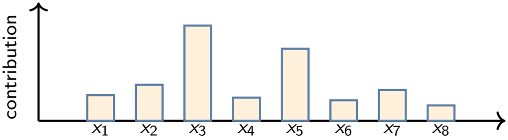
$$e_j^2.$$

Large values show variables that are poorly reconstructed by the PCA model.

T^2 contribution

Score-space contributions ask which original variables pushed the point far from the normal score region.

This is useful, but interpretation must consider correlations between variables.



PCA: Strengths and Limitations

Strengths

- simple and fast;
- useful visualisation tool;
- removes linear redundancy;
- gives ordered components.

Limitations

- only linear projections;
- sensitive to scaling;
- components may be hard to interpret;
- large variance is not always task-relevant.

Practical message

PCA is often a strong first tool for inspecting high-dimensional data, but it is not the only possible representation.

From PCA to Autoencoders

PCA learns a **linear** low-dimensional representation.

An autoencoder learns a representation using a neural network.

$$\boldsymbol{x} \xrightarrow{\text{encoder}} \boldsymbol{z} \xrightarrow{\text{decoder}} \hat{\boldsymbol{x}}.$$

input $\boldsymbol{x}$ → encoder g_ϕ → latent code $\boldsymbol{z}$ → decoder h_θ → output $\hat{\boldsymbol{x}}$

Training signal

The target is the input itself. The network learns by trying to reconstruct its own input.

Autoencoder Objective

The encoder maps input to a latent code:

$$\boldsymbol{z}_i = g_\phi(\boldsymbol{x}_i).$$

The decoder maps the latent code back to the input space:

$$\hat{\boldsymbol{x}}_i = h_\theta(\boldsymbol{z}_i).$$

A common reconstruction loss is

$$J(\phi, \theta) = \sum_{i=1}^{n} \|\boldsymbol{x}_i - h_\theta(g_\phi(\boldsymbol{x}_i))\|_2^2 .$$

Interpretation

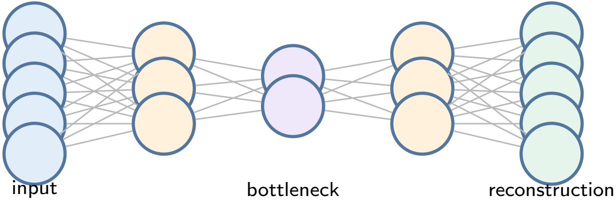
The latent code $\boldsymbol{z}_i$ should keep the information needed to reconstruct the input.

The Bottleneck Idea

If the latent dimension is smaller than the input dimension,

$$\dim(\boldsymbol{z}) < \dim(\boldsymbol{x}),$$

then the network cannot simply copy every input coordinate directly.



Key idea

A good bottleneck representation captures the essential factors of variation in the data.

When Can an Autoencoder Cheat?

If the latent layer is too large and the network is powerful, it may simply learn an identity map:

$$\hat{\boldsymbol{x}} \approx \boldsymbol{x} \quad \text{without learning a useful representation.}$$

Ways to avoid this include:

- use a narrow bottleneck;
- add regularisation;
- add noise and train a denoising autoencoder;
- use sparsity penalties;
- evaluate whether the latent variables are useful for the intended task.

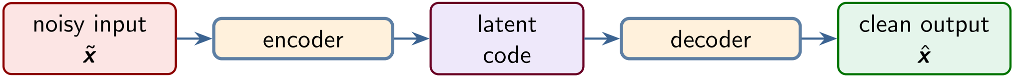
Important

Low reconstruction error alone does not guarantee that the latent representation is meaningful.

Denoising Autoencoder

A denoising autoencoder receives a corrupted input but is trained to reconstruct the clean input.

$$\tilde{\boldsymbol{x}} = \boldsymbol{x} + \text{noise}, \qquad \tilde{\boldsymbol{x}} \longmapsto \hat{\boldsymbol{x}} \approx \boldsymbol{x}.$$



Interpretation

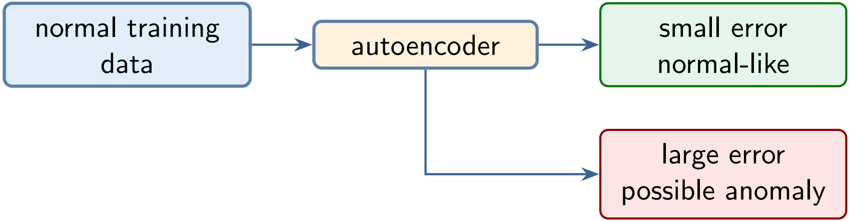
The model learns structure that is stable under noise, rather than memorising every small fluctuation.

Autoencoders for Anomaly Detection

Train an autoencoder on normal data.

For a new observation, compute reconstruction error:

$$r(\boldsymbol{x}) = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2.$$



Practical message

Large reconstruction error can indicate a possible anomaly; the threshold still needs validation.

PCA Monitoring and Autoencoder Monitoring

PCA process monitoring and autoencoder anomaly detection use the same basic logic.

	PCA monitoring	**Autoencoder monitoring**
Representation	linear score vector $\boldsymbol{t}$	nonlinear latent vector $\boldsymbol{z}$
Reconstruction	$\hat{\boldsymbol{x}} = V_q \boldsymbol{t}$	$\hat{\boldsymbol{x}} = g(f(\boldsymbol{x}))$
Residual metric	$Q = \lVert \boldsymbol{x} - \hat{\boldsymbol{x}} \rVert^2$	reconstruction error
Latent-space metric	$T^2 = \boldsymbol{t}^\top \Lambda_q^{-1} \boldsymbol{t}$	distance in latent space

Bridge

PCA is the linear, interpretable starting point. Autoencoders extend the same monitoring idea to nonlinear representations.